

AI Agent 框架搭建 — 实验报告

选题：AI Agent 框架搭建 —— 从“会说话”到“能做事”

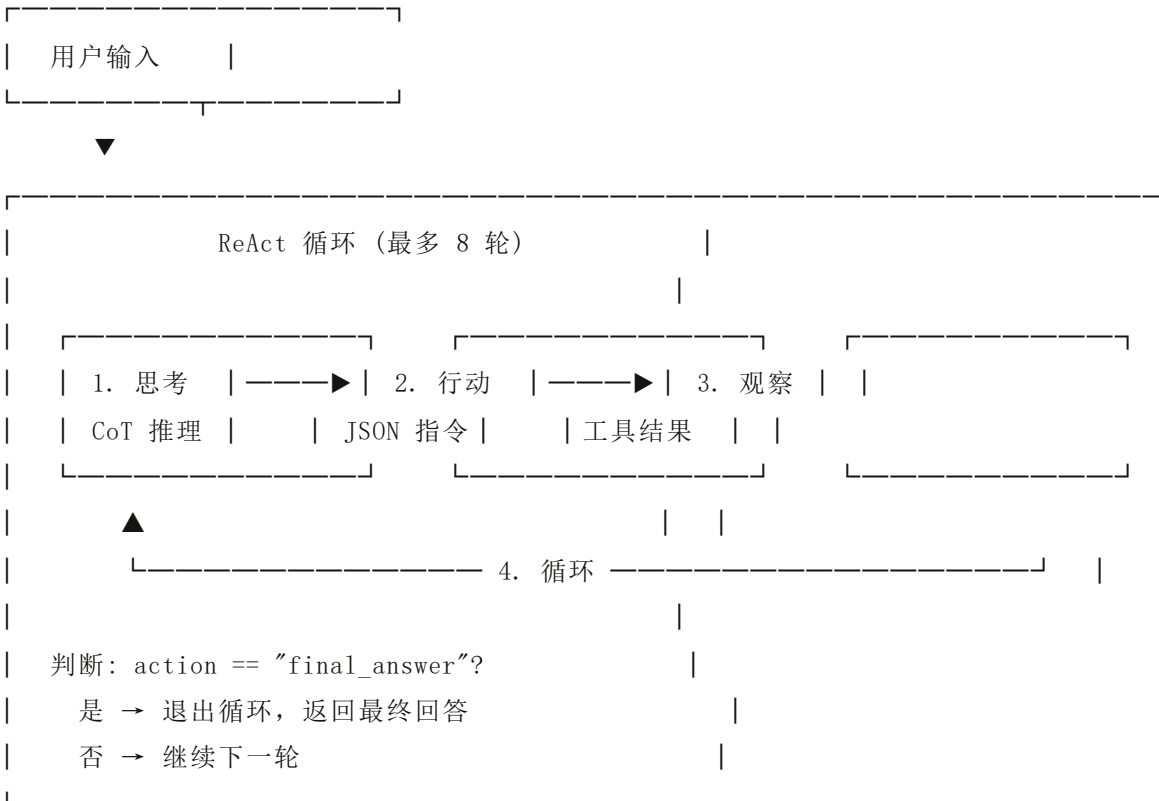
一、核心原理与算法设计

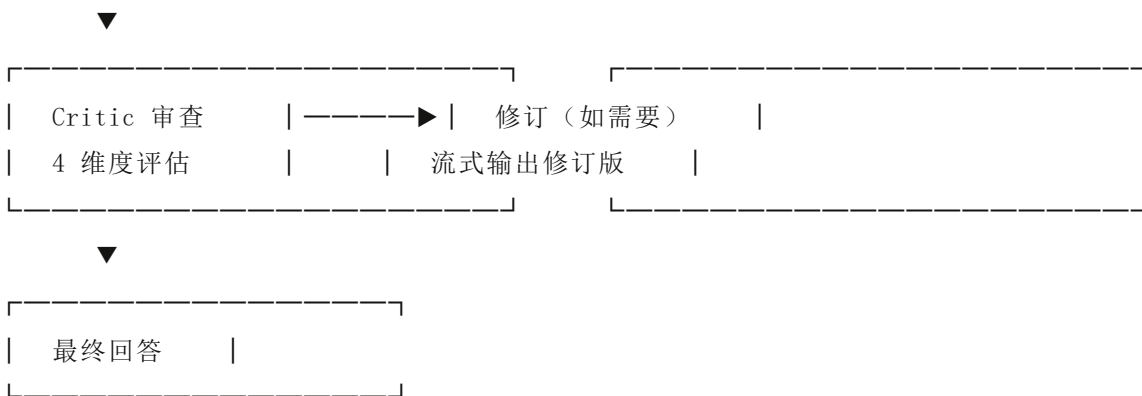
1.1 ReAct 范式

本项目的核心是 ReAct (Reasoning + Acting) 范式：LLM 交替进行“思考”和“行动”，在循环中自主决策、调用工具、观察结果，直到完成任务。

用户输入 → [思考] → [行动：调用工具] → [观察：工具结果] → [思考] → ... → [最终

1.2 Agent 决策闭环流程图





1.3 ReAct 系统提示词模板

你是一个 AI 智能助手，你可以使用工具来解决用户的问题。

当前时间：{动态时间}

可用工具

- calculate: 执行数学计算。参数: expression (字符串)
- search_web: 搜索互联网获取实时信息。参数: query (字符串)
- get_weather: 查询城市实时天气。参数: city (字符串)
- read_file: 读取文件内容。参数: filename (字符串)
- write_file: 写入内容到文件。参数: filename, content (字符串)
- list_files: 列出工作目录下的文件。参数: path (字符串)
- run_python: 在沙箱中执行 Python 代码。参数: code (字符串)

工作方式

1. 思考: 分析用户问题，决定是否需要使用工具
2. 行动: 输出 JSON 格式的工具调用指令
3. 观察: 等待工具执行结果
4. 循环: 根据结果决定下一步

输出格式

调用工具: {"action": "工具名", "args": {"参数名": "参数值"}}

最终回答: {"action": "final_answer", "content": "回答内容"}

规则

- 每次只调用一个工具
- 必须输出合法的 JSON
- JSON 中字符串需转义双引号和换行符

- 不要编造数据，优先使用工具获取信息
- 工具调用失败后反思原因，换策略重试

1.4 ReAct 主循环伪代码

```
def run_agent_stream(user_message, conversation_history):
    messages = build_initial_messages(user_message, history)

    for round_num in range(MAX_ROUNDS): # 最多 8 轮
        # 1. 流式调用 LLM
        response_text = ""
        for token in call_llm_stream(messages):
            response_text += token
            yield {"type": "token", "content": token}

        # 2. 解析 JSON 指令
        action = parse_action(response_text)

        if action is None:
            # 无法解析 → 视为最终回答
            yield {"type": "final_answer", "content": response_text}
            break

        if action["action"] == "final_answer":
            yield {"type": "final_answer", "content": action["content"]}
            break

        # 3. 执行工具
        tool_name = action["action"]
        tool_args = action["args"]
        yield {"type": "tool_call", "name": tool_name, "args": tool_args}

        result = TOOL_REGISTRY[tool_name](**tool_args)
        yield {"type": "tool_result", "content": result}

        # 4. 反思检测
        if is_tool_error(result):
            messages.append(f"工具返回错误: {result}")
            messages.append("请反思失败原因，换一种方式重试。")
        else:
```

```
messages.append(f"工具返回: {result}")

yield {"type": "done"}
```

1.5 Critic Agent 协作机制

主 Agent 输出回答后，Critic Agent（独立 LLM 调用）从 4 个维度审查：

维度	检查内容
事实准确性	数据、事实是否有误？
逻辑完整性	推理是否有漏洞或跳跃？
遗漏	用户问题中是否有未被回答的部分？
改进建议	回答是否可以更清晰、更完整？

Critic 发现问题时，主 Agent 根据审查意见自动修订回答。

二、工程实现细节

2.1 项目架构

```
ai-agent/
├── app.py           # Gradio Web UI（界面布局 + 事件绑定 + 自定义CSS）
├── agent.py         # ReAct 循环引擎 + Critic Agent
├── llm_api.py       # DeepSeek API 封装（SSE 流式 + 非流式）
├── tools.py         # 7 个工具实现 + 注册表 + 安全沙箱
├── style.css        # 自定义 CSS（Claude 暖色系，~620行）
├── requirements.txt # 依赖：gradio, requests, python-dotenv
└── .agent/          # 长期记忆 + 经验库持久化
```

2.2 模块说明

llm_api.py — API 封装

函数	功能	关键参数
<code>call_llm()</code>	非流式调用，返回完整响应	<code>temperature=0.1</code>
<code>call_llm_stream()</code>	SSE 流式调用，逐 token yield	<code>stream=True, timeout=60</code>

- 模型: DeepSeek-V3 (`deepseek-chat`)
- `max_tokens`: 8192
- JSON 序列化使用 `ensure_ascii=True` 解决 Windows surrogate 字符导致的 400 错误

agent.py — ReAct 循环引擎

函数	功能
<code>build_system_prompt()</code>	构建系统提示词（工具描述 + 规则 + 长期记忆 + 经验）
<code>parse_action()</code>	三层 JSON 解析（代码块 → 直接解析 → 括号计数）
<code>run_agent_stream()</code>	主 ReAct 循环生成器，yield 结构化事件
<code>manage_memory()</code>	对话摘要压缩（超过 16 条触发，保留 6 条）
<code>review_answer()</code>	Critic 审查——4 维度评估主 Agent 回答
<code>revise_answer_stream()</code>	根据 Critic 反馈流式修订回答
<code>_summarize_conversation()</code>	调用 LLM 将旧消息压缩为 ≤200 字摘要
<code>_save_experience()</code> / <code>_load_experience_text()</code>	失败经验持久化与注入

JSON 解析策略（三层）：

1. 代码块匹配：正则提取 ````json {...} ````，括号计数法处理嵌套
2. 直接解析：尝试 `json.loads` 整段响应
3. 括号计数法：找到第一个 `{`，逐字符计括号深度，匹配闭合 `}`

```
def _extract_json(text):
    start = text.find("{")
    if start == -1: return None
```

```

depth = 0
for i in range(start, len(text)):
    if text[i] == "{": depth += 1
    elif text[i] == "}":
        depth -= 1
    if depth == 0:
        return text[start:i+1]
return None

```

思考过程与最终回答分离：

Agent 流式输出中，CoT 思考文本（"思考：..."、JSON 指令）与最终回答通过事件分流：- token 事件 → 进入 thinking_buf 缓冲区 - tool_call / tool_result → 进入 thinking_steps[] 列表 - final_answer → clean_answer 独立存储 - UI 渲染：思考过程在 <details> 可折叠组件中，最终回答作为干净 Markdown 显示

自反思机制： - 检测工具返回中的 8 个中文错误关键词（错误/出错/失败/超时/不存在/未找到/状态码/没有权限）
- 触发反思：将"工具返回: {result}"替换为反思指令 - 经验持久化到 .agent/experience.json，注入后续系统提示词

长期记忆（摘要压缩）： - 对话超过 16 条触发 → LLM 压缩最旧 10 条为 ≤200 字摘要 - 持久化到 .agent/memory.json，最多保留 20 条 - 重启后自动加载，跨会话保留

tools.py — 工具模块

工具	实现方式	安全措施
calculate	eval() + __builtins__ 白名单	限制 __builtins__
search_web	三引擎级联：360 → 搜狗 → 必应	频控、Cookie、反爬检测、词典过滤
get_weather	wttr.in API	—
read_file	open()	_safe_path() 限制在 workspace/
write_file	open()	_safe_path() 限制在 workspace/
list_files	os.listdir()	_safe_path() 限制在 workspace/
run_python	exec() + 沙箱	__import__ 白名单（24 模块），10s 超时，stdout 捕获

搜索工具——三引擎级联：

360搜索 (so.com) ——主引擎——→ 中文查询首选
↓ 失败
搜狗 (sogou.com) ——备选——→ Session 复用, 403 重建
↓ 失败
必应 (bing.com) ——兜底——→ 30+ 词典特征词过滤
↓ 失败
返回“未找到相关信息”

app.py — Web UI

功能	实现
界面框架	Gradio 6.x, gr.Blocks
布局	侧边栏 (240px) + 主面板 (flex: 1)
事件处理	gr.on() + .then() 链式调用
流式渲染	respond() 生成器 yield (history, agent_history)
文件树	递归遍历 workspace/, 树状连接线 (—└—)
文件预览	点击文件 → 弹出面板 (opacity + 滑入动画)
自动滚动	requestAnimationFrame 持续检测 scrollTopHeight
输入清空	.then(lambda: "", None, msg) 响应后清空

2.3 关键数据处理

对话历史格式：

```
agent_history = [  
    {"role": "user", "content": "我叫小明"},  
    {"role": "assistant", "content": "你好小明！"},  
    ...  
]
```

- 存储干净消息对（不含工具调用标记）

- 短期记忆：保留最近 10 轮 (20 条)
- 传给 LLM 时前置到当前对话之前

记忆压缩流程：

```
对话 > 16 条 → manage_memory()  
    → _summarize_conversation(最旧 10 条) → LLM 压缩为摘要  
    → _save_memory(摘要) → .agent/memory.json  
    → _load_memory() → 注入下次 system prompt
```

三、实验结果与分析

3.1 多步推理对话示例

任务：搜索量子计算进展并保存到文件

[用户] 搜索量子计算最新进展并保存到 quantum.txt

[思考过程 — 第 1 轮]

思考：用户需要搜索量子计算的最新进展。我应该先使用 search_web 工具来获取信息。

调用工具：search_web

参数：{"query": "量子计算最新进展 2026"}

[工具返回]

1. IBM 发布 1000+ 量子比特处理器，纠错能力大幅提升
2. Google Quantum AI 实现容错量子计算里程碑
3. 中国科大实现量子优越性新纪录

[思考过程 — 第 2 轮]

思考：已获取量子计算的最新进展信息。现在需要将这些内容写入 quantum.txt 文件。

调用工具：write_file

参数：{"filename": "quantum.txt", "content": "量子计算最新进展 (2026):\n1. IBM..."}

[工具返回]

文件 quantum.txt 写入成功

[思考过程 — 第 3 轮]

思考：信息已成功搜索并保存到文件。现在可以给出最终回答。

[最终回答]

已搜索量子计算最新进展并保存到 `quantum.txt`。主要包括：

1. IBM 发布 1000+ 量子比特处理器，纠错能力大幅提升
2. Google Quantum AI 实现容错量子计算里程碑
3. 中国科大实现量子优越性新纪录

分析： Agent 在 3 轮内完成了“搜索 → 写入 → 总结”的完整流程。第 1 轮正确选择搜索工具，第 2 轮将搜索结果写入文件，第 3 轮给出最终回答。整个过程展示了 ReAct 循环的完整决策链。

3.2 多步推理对话示例 2（自反思）

任务：读取不存在的文件

[用户] 读取 `hehe.txt` 文件的内容

[思考过程 — 第 1 轮]

调用工具：`read_file`

参数：`{"filename": "hehe.txt"}`

[工具返回]

错误：文件 `hehe.txt` 不存在

[反思提示] 请反思失败原因，换一种方式重试。

[思考过程 — 第 2 轮]

思考：读取 `hehe.txt` 失败，该文件不存在。我先列出目录看看有哪些文件。

调用工具：`list_files`

参数：`{"path": ""}`

[工具返回]

`workspace/` 下的文件：

`quantum.txt, weather.txt, test.txt, primes.txt`

[最终回答]

`hehe.txt` 不存在。`workspace` 中现有文件：`quantum.txt, weather.txt, test.txt, primes.tx`

分析： Agent 在工具调用失败后触发了自反思机制。第 1 轮读取失败，第 2 轮换策略改用 `list_files`，最后告知用户文件不存在并展示可用文件。验证了反思 prompt 的有效性。

3.3 Critic Agent 协作示例

任务： 用户问"我叫什么名字？"（前文提到叫小明）

[主 Agent 回答]
你叫小明。

[Critic 审查]
回答质量良好，无需修改。（因为有对话历史上下文，知道"小明"来自用户之前输入）

对比：无 Critic 上下文时：
[Critic 审查]
回答缺乏事实依据——用户未在当前对话中提供姓名，"小明"系编造。
建议改为：我无法确定你的名字，你可以告诉我。

分析： Critic Agent 在拥有对话历史上下文时，能正确判断"小明"来自用户之前的陈述而非编造。验证了 Critic 审查的有效性和上下文注入的必要性。

3.4 关键参数对比

参数	旧值	新值	效果
max_tokens	2048	8192	支持长代码/长文件写入
temperature	0.7	0.1	减少随机性，JSON 格式更稳定
最大 ReAct 轮次	5	8	支持更复杂的多步任务
短期记忆轮数	—	10	新增短期记忆
搜索引擎	1 (必应)	3 (360/搜狗/必应)	中文搜索质量大幅提升
工具数量	3	7	新增 run_python、天气、文件操作

3.5 系统资源消耗

指标	典型值
单次对话 API 调用次数	2-5 次（含 ReAct 循环）

指标	典型值
Critic 审查额外调用	1-2 次
平均响应延迟	3-8 秒（含工具执行）
内存占用	~80MB（Python 进程）
磁盘占用	源代码 ~80KB，workspace 按需增长

四、遇到的问题与解决方案

问题	原因	解决方案
API 请求 400 错误	Windows JSON 序列化产生 surrogate 字符	<code>ensure_ascii=True</code> + byte 编码
搜索工具超时/不可用	百度/维基百科国内受限	三引擎级联：360→搜狗→必应
Agent 无限循环	LLM 不输出 JSON	<code>parse_action=None</code> 视为 <code>final_answer</code>
长文件写入失败	<code>max_tokens</code> 不够 + JSON 截断	8192 tokens + 括号计数法
聊天页面整体滚动	主面板 <code>overflow-y: auto</code>	flex 隔离：仅 chat-main 可滚动
CoT 思考遮挡答案	所有 token 混入气泡	事件分流 + <code><details></code> 折叠组件
Agent 破坏项目文件	<code>run_python</code> 无 import 限制	<code>__import__</code> 白名单机制
工具失败后盲目重试	无失败检测	自反思机制 + 经验库持久化
长期对话丢失上下文	短期记忆超限直接丢弃	摘要压缩 + <code>memory.json</code> 持久化

问题	原因	解决方案
气泡多层重叠	Gradio 6.x 双层卡片	Playwright DOM 排查 → 逐层透明
裸 JSON 泄漏到回答	strip_json 仅匹配代码块	括号计数法 + try_extract_answer 兜底
Critic 误判记忆回答	无对话历史上下文	conversation_history 注入 Critic prompt
搜索返回词典释义	Bing 中文索引差	30+ 词典特征词过滤

五、总结

本项目从零搭建了一个完整的 AI Agent 框架，实现了：

- 1. **ReAct 循环引擎**（最多 8 轮自主决策）
- 2. **7 个工具**（计算/搜索/天气/文件读写/目录/Python 沙箱）
- 3. **流式输出**（SSE 逐 token 渲染 + 思考过程可折叠）
- 4. **短期记忆**（10 轮上下文）+ **长期记忆**（摘要压缩 + 跨会话持久化）
- 5. **自反思**（失败检测 + 经验库 + 换策略重试）
- 6. **多 Agent 协作**（Critic 审查 + 自动修订）
- 7. **Web UI**（Gradio + Claude 暖色系 + 文件树可视化 + 弹出预览）
- 8. **安全沙箱**（路径限制 + `__import__` 白名单 + 超时控制）

技术栈：Python + DeepSeek-V3 API + Gradio 6.x，未使用任何第三方 Agent 框架。

通过本项目，深入理解了 LLM 应用层最核心的设计模式——ReAct 范式、Prompt 工程、工具编排、记忆管理和多 Agent 协作。